



山东大学
SHANDONG UNIVERSITY

WaterOS 内核设计与优化实现文档

参赛队名: OuterSystems

队伍成员: 宋浩宇、李佳灿、孙馨宇

指导老师: 颜廷坤、潘润宇

目 录

第 1 章	我们为什么开始做这轮优化	1
1.1	优化问题的分层	1
1.2	实验分支提供了什么	2
1.3	最终纳入的优化范围	2
第 2 章	基线、分析工具和取舍标准	3
2.1	先固定什么	3
2.2	用 pc-hot 看指令花在了哪里	3
2.3	用 syscall profile 解释调用形态	4
2.4	从热点到提交的判断链	4
第 3 章	我们保留下来的核心优化路径	5
3.1	系统调用入口：减少重复导入和查表	5
3.2	VFS 与文件系统：让解析结果真正可复用	5
3.3	exec 与只读文件页：从重复读取走向物理页共享	6
3.4	按实际页表变化进行 TLB 同步	6
3.5	任务、futex 与每 CPU 快路径	7
3.6	把物理页清零移出繁忙 CPU	7
第 4 章	结果、反例和数据边界	8
4.1	各项实验分别说明了什么	8
4.2	pc-hot 如何帮助判断局部优化	8
4.3	零页池的机制数据	9
4.4	没有纳入最终组合的实验	9
4.5	与 Linux 基线的距离	10
4.6	结果的适用边界	10
第 5 章	AI 使用说明与复现实验	11
5.1	AI 在开发过程中的位置	11
5.2	复现环境与证据分级	11
5.3	pc-hot 与 syscall profile 的复现方法	12
5.4	完整 BuildStorm A/B	12
5.5	文档和实现如何对应	13
第 6 章	总结与下一步工作	14

第 1 章 我们为什么开始做这轮优化

这份文档记录的是为了跑通并加速综合编译负载，对 WaterOS 内核进行的一轮实际优化。优化对象不是某一个孤立函数，而是一条贯穿用户程序、系统调用、任务调度、虚拟内存、VFS、文件系统和块设备的长路径。单次调用中的一次查表、一次页表刷新或一次内存清零并不起眼，但 BuildStorm 会重复创建进程、装载 ELF、查询工具链文件并申请匿名页，这些固定成本最终会累计成数百秒的差距。

仓库中的 perf/* 分支保留了这轮工作的实验轨迹。分支既包括地址空间缓存、条件 TLB 刷新、只读页共享和缓存准入等有效方案，也包括扩大缓存、修改调度周期、重写 LRU 或增加专用小对象池等失败实验。我们没有把“做过”直接等同于“有效”：正文以已经进入主线、或具有完整 A/B 证据的方案为主；未获得端到端收益的实验仍保留，用来解释为什么没有继续沿那个方向投入。

整个过程可以概括为三件事。先把运行环境和完成标记固定下来，保证两次结果确实可比；再通过 pc-hot、wait-hot 和 syscall profile 从完整负载中提取指令、等待和参数分布；最后只改变一个主要因素做交错 A/B，并在功能回归通过后决定保留还是回退。这样做比直接改数据结构慢一些，却能区分“符号自身变冷”和“用户真正更快”这两件事。

1.1 优化问题的分层

本轮工作逐步收敛为四层问题。第一层是重复工作，例如同一路径被多次解析、同一 fd 在一条系统调用链中反复进入 registry。第二层是缓存策略，例如工具链只读页的跨进程复用，以及低命中块缓存是否值得接纳第一次访问。第三层是同步粒度，例如 lazy VMA 没有产生 PTE 时仍进行全地址空间 TLB 刷新。第四层是工作迁移，即把必须完成、但不必由繁忙 CPU 同步完成的页面清零转移到 idle CPU。

这些层次之间有先后关系。若调用链仍在重复查找，扩大缓存只会掩盖上层浪费；若缓存对象缺少稳定身份，跨进程共享就无法保证失效语义；若没有先测量 PTE 是否变化，简单删除 fence 又会破坏正确性。因此，后续章节按“测量—消除重复—改善复用—缩小同步—迁移前台工作”的顺序展开，而不是按提交日期罗列。

1.2 实验分支提供了什么

表 1-1 性能实验分支的主要类别

类别	代表性方向	在本文中的用途
测量工具	syscall profile、cache diagnostics、ELF page profile	先确认调用次数、参数形态、命中率和复用距离，再选择优化对象。
已保留优化	当前地址空间缓存、单页 COW 刷新、mprotect 变化摘要、只读 mmap/ELF 页缓存	构成地址空间、TLB 和文件页复用的主要实现路径。
策略实验	block-cache admission、page-cache capacity、negative dentry、readahead	用完整 A/B 确定容量、准入和淘汰策略，而不是依赖经验参数。
回退实验	40/48 MiB 页缓存、128 B TLSF 固定池、调度周期修改、intrusive LRU	记录无收益、退化或正确性问题，防止重复尝试同一路径。

1.3 最终纳入的优化范围

表 1-2 正文覆盖的最终优化主线

方向	最终保留的工作
测量与基线	固定 QEMU 9.2.1、镜像、CPU 数量和 -snapshot，建立完整 Build-Storm 与短窗口诊断两级方法。
syscall 与 FD	批量导入用户元数据，复用已分类 socket 和 FD I/O lease，减少用户地址空间及 registry 的重复访问。
VFS 与文件系统	复用路径解析结果，缓存正负 dentry，改进缓存准入，并为稳定只读访问建立明确契约。
exec 与内存映射	共享只读 ELF/可执行 mmap 物理页，复用 exec 文件句柄，并按真实 PTE 变化决定 TLB 同步。
任务与同步	保留当前地址空间快路径、单页 COW 失效和 futex registry 分片等能够证明语义边界的改动。
物理页分配	由 idle CPU 预先清零页面并放入零页池，前台 fault 和页表分配优先消费已清零页面。

第 2 章 基线、分析工具和取舍标准

2.1 先固定什么

我们把“能不能跑完”和“跑得快不快”分开记录。前者由功能测例、构建门禁和完整阶段标记确认；后者使用同一宿主、同一 QEMU、同一镜像和同一输入做交错 A/B。完整 BuildStorm 的计时从 workload 的统一起点开始，到内核报告 BUILDSTORM_COMPILE mode=multi ok=true elapsed_s 为止。没有结束标记、出现 panic 或只跑完前置阶段的记录，都不能作为性能结果。

正式实验使用 QEMU 9.2.1、RISC-V virt、8 个 vCPU、16 GiB 内存、发布配置内核和同一份镜像，并始终带 -snapshot。每轮至少记录内核提交、内核与镜像哈希、QEMU 参数、宿主 CPU 亲和性、开始与结束时间。涉及架构无关代码时执行 RISC-V 与 LoongArch 静态检查；涉及 MM、文件系统、task 生命周期的改动还要构建 final kernel 并执行相应功能回归。

宿主调度是不能忽略的变量。一次对照中，同一工作负载落在性能核时约为 1348.86 秒，而落在效率核时约为 1881.13 秒，差距约 28%。这不是内核优化，因此本地 A/B 必须固定宿主 CPU 集合，并交错运行 A-B-A 或 B-A-B。低于宿主漂移范围的变化只记为“无可确认收益”。

2.2 用 pc-hot 看指令花在了哪里

pc-hot（部分记录中简称 pchot）是基于 QEMU TCG plugin 的 PC 采样工具。插件对翻译执行的 guest PC 计数，退出时一次写出结果；分析脚本再使用带符号的内核 ELF，将 PC 聚合到 Rust 函数。它不需要在内核热路径加入日志，因此适合比较候选前后的指令分布。

一个典型流程如下：

1. 使用与被测内核匹配、保留符号的 final ELF；
2. 构建对应架构的 pc-hot 插件；
3. 在固定 300 秒窗口内运行相同 workload，并让 QEMU 正常退出或由 timeout 结束；
4. 用 nm 和 addr2line 聚合前若干热点，比较总指令以及各符号占比；
5. 若使用 -icount shift=N，分析时传入同一 shift，查看每个 vCPU 的虚拟时间分布。

早期 K32 窗口中，memcpy、memcmp 和 memset 分别约占 35.6 亿、30.7 亿和 18.3 亿条指令；随后是 VirtQueue、TLSF 分配、缺页、块缓存读取和缓存淘汰。这个结果没有直接给出“应该优化 memcpy”的结论。它说明大量成本来自上层重复拷贝、键比较、页面清零和块读取，真正需要检查的是这些基础函数为什么被调用了这么多次。

完整 wait-hot 记录进一步采样了约 2810.5 亿条指令，其中 mprotect、memcpy、memset、

page fault 和 scheduler 都进入前列。各 vCPU 的 idle 时间明显不均：有的核心仅空闲约 629 秒，有的超过 1200 秒。这表明 BuildStorm 的关键阶段仍受一个或少数繁忙核心限制，也为后续“由 idle CPU 预清零页面”提供了依据。

表 2-1 pc-hot 与 wait-hot 分别回答的问题

工具	主要输出	适合回答的问题
pc-hot	PC、函数及每个 vCPU 的指令数	哪些内核路径消耗了指令；候选是否只是把成本移到另一个符号。
wait-hot	idle 时间、阻塞系统调用墙钟分布	是计算热点、锁等待、I/O 等待，还是核心负载不均造成停顿。
syscall-profile	syscall 次数、参数桶、路径复用、flags 热点	workload 实际如何使用 Linux ABI，应该优先优化哪一种调用形态。

2.3 用 syscall profile 解释调用形态

只看符号仍可能误判。例如 mprotect 很热，但不知道它是在修改大段驻留页，还是反复修改尚未驻留的 lazy VMA。为此我们增加了 syscall profile 插件。WaterOS full-system 模式在 RISC-V 用户态 ecall 处读取 a7 与 a0.a5，按 vCPU 聚合 syscall、参数数量级、flags 和有界路径字符串；运行期不逐条打印，也不修改 guest 内核。

300 秒 BuildStorm 画像共捕获 340,830 次 syscall。mprotect 为 100,154 次，占 29.39%；其中 85,544 次长度位于 4 KiB 桶，98,643 次目标权限为 PROT_READ|PROT_WRITE。同一窗口内，statx、openat 和 readlinkat 的成功路径重复比例分别约为 81.99%、69.90% 和 92.14%。前一组数据指向 lazy VMA 的条件 TLB 同步，后一组数据则证明工具链和 Cargo registry 的路径、metadata 确实被反复访问。

插件数据只用于诊断。ecall 后端尚不能在 full-system 中可靠取得 syscall return callback，因此 read/write 统计的是请求长度而不是实际返回长度；固定窗口里执行的 guest 工作量也可能不同。它可以决定下一步实验，却不能代替完整 BuildStorm 墙钟成绩。

2.4 从热点到提交的判断链

我们的判断过程固定为五步：先在完整日志中确定阶段和完成状态；再用低开销统计确认调用次数、锁等待、缓存命中或页表变化；随后提出一个能够解释数据的机制假设；实现只改变该假设所需的最小边界；最后回到完整 workload 做交错 A/B。若只降低了某个函数指令数，却没有改善完整耗时，就回退实现并保留记录。

这套标准也限定了优化的风险。查询路径不能持有 registry 锁跨越用户拷贝、阻塞或调度；fd 复用不能越过 close/reuse 与文件偏移的线性化点；页表同步只能在接口明确返回变化摘要后缩小；缓存必须说明对象身份、容量、失效和回收边界。性能收益不能用来抵消 ABI、持久化或并发语义的破坏。

第 3 章 我们保留下来的核心优化路径

3.1 系统调用入口：减少重复导入和查表

syscall 本身的分发表不是主要热点，真正的固定成本通常出现在参数进入内核以后。早期实现中，poll/select 每次重新扫描都会从用户空间复制监视集合；路径字符串按单字节访问用户页；iovec 元数据逐项复制；socket readiness 与 nonblocking 判断也可能对同一个 fd 重复分类。这些操作单次很短，却被等待循环和高频 I/O 放大。

我们先处理不改变 ABI 和线性化点的部分。poll/select 在入口导入一次不可变监视集合，等待期间只重新查询设备状态；路径字符串改为按小块复制，跨页错误时再退回逐字节处理，以保留 NUL、EFAULT 和长度边界；readv/writev 等将连续 iovec 元数据整体导入，但 vmsplice 仍保留逐项读取，因为提前导入会改变后续 fault 时的部分成功语义。socket 在单次扫描中只分类一次，实际可能阻塞的 I/O 仍使用 detached handle，不持有 fd registry 锁进入睡眠。

随后，FD slot 保存稳定的资源分类和 flags 快照，read/pread 家族通过 I/O lease 在一次操作内复用查询结果。这个 lease 只覆盖能够证明安全的临界范围：它不跨越 close/reuse 的 fd 代际，不把文件偏移更新移出原有线性化点，也不为了减少一次 clone 而删除阻塞路径所需的 detached handle。这样取得的收益来自“少做一次相同工作”，而不是削弱并发语义。

3.2 VFS 与文件系统：让解析结果真正可复用

BuildStorm 会反复访问 Rust sysroot、Cargo registry、target 描述和同一构建目录。syscall 画像已经证明 statx、openat 和 readlinkat 的路径具有很高重复率。原调用链却可能在 syscall、VFS bridge 和具体文件系统之间重复拆分路径、查询 mount、读取 metadata，再以字符串 key 进入页缓存。

优化后的方向是让一次 lookup 产生可消费的稳定结果。解析结果携带节点身份、metadata 和 mount identity，openat、statx 以及后续 read 不必重新从字符串开始。mount namespace 使用 Arc/COW 快照，普通读取不再深拷贝整份命名空间；正 dentry 采用有界淘汰，负 dentry 记录确定的 NotFound，在目录发生创建、重命名等改变时失效。对根文件系统的稳定只读操作，则建立不依赖全局可变 FS 锁的只读访问契约。

缓存策略来自访问数据，而不是“越大越好”。块缓存画像记录了 15,530,833 次读取，但只有 211,798 次命中，命中率约 1.36%；第一次 miss 后再次访问的 ghost revisit 只有约 0.51%。因此第二次命中才准入的策略避免让一次性编译流量污染有限缓存，完整候选由 797.21 秒降到 783.00 秒。相比之下，文件页缓存命中约 56%，更值得承载工具链的重复只读页。

页缓存优化也包括稳定 key 和生命周期处理。复用 file key 后，构造 key 的指令由约 1151 万降到 353 万；关闭文件时不立即全量清除干净页，使 `purge_closed_file` 从约 3.50 亿条指令降到接近零。这里仍然保留 `truncate`、`rename`、写回和文件身份变化的失效边界，不能把“读路径稳定”扩展成“永不失效”。

3.3 exec 与只读文件页：从重复读取走向物理页共享

文件路径收敛后，热点继续进入 `exec`、ELF 装载和 `file-backed fault`。多个 `rustc` 子进程会装载同一工具链 ELF 和只读映射；若每个进程都重新分配物理页、清零并从文件复制，VFS 命中只能省去磁盘访问，不能消除内存端的重复工作。

我们首先让 `exec` 前缀识别、ELF 头和 `PT_LOAD` 解析复用同一个稳定文件句柄，并在双架构上保留 lazy ELF 的差异处理。随后对只读 ELF 页建立以稳定文件身份和页偏移为 key 的物理页缓存。一次 profile 中共有 40,960 次可共享只读 ELF fault，其中只有 7,325 个唯一页 key，重复率为 82.11%；仅按 4 KiB 页面估算，重复 fault 至少对应约 131.39 MiB 可避免的清零与复制。

进一步的只读可执行 `mmap` 缓存把同一原则扩展到 ELF loader 之外。64 MiB 配置的完整候选为 640.95 秒；扩大到 128 MiB 后为 534.26 秒，在该组独立 A/B 中改善 16.65%。继续增至 192 MiB 并未证明更好，因此容量仍由工作集测量决定。缓存只接受内容稳定、权限只读、文件身份和偏移明确的页；私有可写映射、文件变化和 COW 继续走原有路径。

3.4 按实际页表变化进行 TLB 同步

`syscall profile` 显示大量 4 KiB `mprotect(RW)`，而 WaterOS 的 `private anonymous mmap` 已经是 lazy VMA。对于尚未驻留的页面，`mprotect` 只改变 VMA 元数据，并没有旧 PTE 可供 CPU 缓存。原实现仍然在 MM 层和 `syscall helper` 中执行全地址空间 flush 与远端 shutdown，造成重复同步。

我们没有直接删除 `fence`，而是修改接口返回“是否实际改变驻留 PTE”的摘要。双架构 `MmapOps::mprotect` 遍历目标范围：未驻留 lazy 页只更新 VMA，相同权限不重写 PTE，权限或 PPN 确实变化时才标记 `changed`。`syscall` 层仅在成功且 `changed` 为真时执行一次本地 flush 和远端 shutdown；错误路径继续保守刷新，因为范围前部可能已经修改而后部才报错。

固定镜像的交错结果为候选 810.26 秒、main 829.67 秒、候选 803.36 秒，候选中位数 806.81 秒，相对夹在中间的 main 改善约 2.76%。同样的原则也用于 `mmap/munmap` 的变化汇总，以及 COW fault 的单页失效。这里优化的是同步范围，而不是省略必要同步。

3.5 任务、futex 与每 CPU 快路径

BuildStorm 会创建大量短生命周期进程，但“进程很多”不等于应该增加更多全局索引。per-parent child index 曾使完整轮退化，current pid 快路径和调度 nice 快路径也没有稳定收益。最终保留的方案集中在调用次数高、且所有权简单的状态上：当前用户地址空间按 CPU 缓存，减少同一 CPU 上的 registry 查询；futex registry 按 key 分片，缩小不相关地址之间的竞争；进程 I/O 统计将同一次双向搬运的计数合并到一次 owner 查找和一次锁内更新。

这些改动都不跨越等待和唤醒边界。futex 分片仍按同一个 FutexKey 匹配 waiter，注册、条件复查和入队的 lost-wake 防护不变；查询路径不会在持锁时访问用户地址、文件系统或调度器。与其追求“完全无锁”，我们更重视缩小共享范围而不改变事件顺序。

3.6 把物理页清零移出繁忙 CPU

当只读文件页能够复用后，匿名页、页表页和零填充映射的前台清零变得更突出。Rust 编译的大 crate 会在单个繁忙 vCPU 上集中触发 fault，而其他 vCPU 有较长 idle 时间。页面必须清零是安全语义，不能删除；但清零不必由请求页面的 CPU 同步完成。

因此我们增加了一个全局固定容量的 LIFO 零页池。idle CPU 在进入 WFI 前尝试批量取得 raw frame，在池锁和 allocator 锁之外清零，再发布到池中；前台 zeroed allocation 先 pop 已清零页，池不足时同步批量补充并保留单页兜底。正式参数为 1024 页容量、256 页低水位、idle 批量 32 页和同步 miss 批量 16 页，最多占用 4 MiB 可回收内存。

普通 allocator 分配 raw frame → producer 独占 → 锁外清零 → zeroed pool
demand: pool 命中直接返回；miss 时同步补充；内存压力下可排空并归还 allocator

图 3-1 预清零物理页的所有权转移路径

并发约束比池本身更重要。一个 PPN 同一时刻只能属于普通 allocator、清零中的 producer、零页池或调用者之一；不同时持有 allocator 锁和 pool 锁；idle 维护拿不到 allocator 锁就跳过，不自旋；已经被写入后释放的页面回普通 allocator，不以“曾经清零”为由回池。池只是缓存，任何时候失效都退回原来的同步清零路径。

256 页统计实验中，demand hit 为 2,158,384，miss 为 29,864，命中率 98.6353%；78.16% 的 refill 页面由 idle CPU 完成，OOM drain 为 0。该结果证明工作迁移确实发生，也说明小池仍会在 fault 突发中达到容量上限，因此正式配置才提高到 1024 页。RISC-V 用户返回路径同时消除了浮点寄存器的重复恢复，避免在高频 trap return 中重复执行 32 次 fld。本章各项优化都保留慢路径和错误路径；缓存未命中或 idle CPU 无法补充时，系统仍回到原有实现。

第 4 章 结果、反例和数据边界

4.1 各项实验分别说明了什么

表4-1 汇总的是若干独立实验,而不是一条可以逐项相加的优化曲线。它们的日期、main 基线和已合入优化不同,因此只能在同一行内部比较。本文保留这些数据,是为了说明每项机制是否把局部观察传递到了完整 workload。

表 4-1 具有完整 BuildStorm 结果的代表性实验

实验	基线耗时	候选耗时	同组结论
mprotect 条件 TLB 同步	829.67 s	中位数 806.81 s	约改善 2.76%，两轮候选均完整通过。
ext4 负 dentry	807.11 s	787.76 s	重复不存在路径可被吸收，单组改善约 2.40%。
block cache 二次命中 准入	797.21 s	783.00 s	一次性块流量不进入有限缓存，约改善 1.78%。
只读 exec mmap cache	783.00 s	640.95 s	跨进程物理页复用在该组改善约 18.14%。
exec mmap cache 64-128 MiB	640.95 s	534.26 s	工作集仍大于 64 MiB，扩容在该组改善 16.65%。
前台清零—idle 预清零	530 s	502 s	代表性完整轮改善 5.28%，仍需以更多交错轮更新中位数。

这些数字也展示了优化重点的变化。最初减少一次 lookup 或一次 fence，收益通常在几个百分点；当画像证明同一只读页被不同进程大量重复建立后，物理页共享才带来两位数改善；继续优化后，匿名页清零又成为繁忙 CPU 上可见的固定成本。热点会随着前一项优化消失而迁移，不能长期依赖同一份 profile。

4.2 pc-hot 如何帮助判断局部优化

pc-hot 最有价值的地方不是给出一个排行榜，而是验证因果关系。保留关闭文件后的干净缓存页后，purge_closed_file 从约 3.50 亿条指令降到约 3 万，总指令下降约 3.6%；复用 FileCacheKey 后，key 构造下降约 69%，TLSF alloc/free 同时下降约 8%。这些变化说明优化确实减少了对对象构造和清理，而不是把成本改名。

但同一工具也会否定看似成功的方案。块缓存内部热点曾从约 6.69 亿条指令降到 5.75 亿，约减少 14%，完整 Final 却仍接近 1957 秒；page-cache key 规范化虽然减少 memcmp，总指令反而增加。我们因此同时查看总指令、候选在固定窗口内推进到的 workload 阶段、相关热点和完整墙钟，不能只截取下降最大的符号。

条件 mprotect 实验是较完整的例子。候选 300 秒窗口执行的 guest 指令比对照多 7.41%，说明相同时间推进了更多工作；request_tlb_shootdown_targets 的归一化占比下降，且没有出现新的 mprotect 热点。这个诊断方向与完整 A/B 一致，因此可以支持、但不替代墙钟结论。

4.3 零页池的机制数据

表 4-2 256 页零页池的独立统计

指标	数值
demand hit / miss	2,158,384 / 29,864
demand hit rate	98.6353%
idle refill	1,710,498 页 (78.16%)
synchronous refill	477,824 页 (21.84%)
峰值 / 最终池长度	256 / 224 (另有 32 页 in-flight)
低水位激活	4,135 次
allocator lock busy 的 idle pass	2,657 次
OOM drain	0 次

98.6% 的 demand 命中说明前台通常能直接取得已清零页；78.16% 的 refill 由 idle CPU 完成，则说明工作确实被迁移，而不是仅靠 demand CPU 批量清零伪装成缓存命中。另一方面，池多次达到上限且仍有 1.36% miss，表明 256 页不足以稳定吸收编译 fault 突发。正式容量提高到 1024 页后，需要用关闭统计探针的内核重新做多轮 A/B；机制统计本身不能推导出新的墙钟百分比。

4.4 没有纳入最终组合的实验

失败实验同样构成结果。它们帮助我们区分 workload 的真实结构和对热点名称的想象。

表 4-3: 代表性回退实验及原因

实验	原本的假设	实际结果与处理
page cache reverse index	避免 purge 时扫描全部 key	局部扫描减少，但总指令和完整耗时没有净收益，回退。
页缓存扩到 40/48 MiB	更大容量应提高命中率	32 MiB 曾有约 5% 改善，继续扩大出现无收益或挂起，不保留。

实验	原本的假设	实际结果与处理
readahead 8 页改 16 页	编译读取具有连续性	完整轮由约 1282.12 s 变为 1321.26 s，一次性流量和额外 I/O 抵消收益。
8-way 页索引	减少字符串 key 的 memcmp	memcmp 下降，但 MINIBUILD 正确性失败，立即回退。
动态 hash bucket	随访问量调整索引规模	1292.96 s 未优于 1282.12 s 对照，变化处于退化方向。
process child index	加速 wait/reap 的子进程查找	新增维护成本使完整轮变慢，说明瓶颈不在单次子节点搜索。
TLSF 小对象固定池/懒统计	减少 allocator 固定开销	局部计数下降但总指令或墙钟没有稳定改善，未进入主线。
intrusive LRU 与满池 bypass	降低只读页缓存淘汰成本	复杂度增加，完整 A/B 不支持继续扩大实现。
timer period 与调度快路径	减少调度进入次数和轻量查询	未形成稳定端到端收益，且可能改变延迟行为，回退。

4.5 与 Linux 基线的距离

一次固定条件对照中，RISC-V Linux BuildStorm 约为 395.90 秒，WaterOS 当时约为 1031.52 秒，约为 Linux 的 2.61 倍。LoongArch 也存在明显差距，但历史记录中的 CPU 数量并不完全一致，不能与 RISC-V 结论直接横向比较。后续多个候选已显著缩短 WaterOS 时间，但这些结果来自不同 main 基线，不能拼接成一条伪造的最终曲线。

当前可以明确的是，重复只读页、前台 fault/清零、VFS 路径与 metadata、TLB shutdown 仍是主要成本；VirtIO 并行度、调度负载均衡以及编译关键进程的单核执行路径仍有空间。下一轮应先在最终提交上重新建立 Linux/WaterOS 双方完整基线，再以同一工具链和相同 CPU 数量报告最终差距。

4.6 结果的适用边界

本文中的 profile 多为 120 或 300 秒固定窗口，用于解释机制；墙钟结论来自带完整成功标记的 BuildStorm。单次 530 秒到 502 秒只作为代表性结果，不替代多轮中位数。不同日期的对照不能相加，QEMU TCG 下的指令变化也不等同于真实硬件周期变化。只有在固定镜像、CPU 亲和性和 QEMU 参数下完成的交错 A/B，才适合用于评价同一候选。

第 5 章 AI 使用说明与复现实验

5.1 AI 在开发过程中的位置

本轮工作使用 AI 辅助阅读跨组件调用链、归纳日志、讨论候选方案、补充定向测试和整理文档。由于性能问题很容易被局部数据误导，AI 输出不能直接作为实现结论。热点是否真实、锁能否缩小、缓存对象是否稳定，以及一项改动是否进入主线，都由源码审查和实测决定。

在 syscall/VFS 优化中，AI 帮助把 syscall 入口、用户拷贝、fd registry、VFS session 和具体文件系统的重复访问连成一条路径，并提示检查阻塞点与文件偏移的线性化边界。在 MM 和零页池优化中，AI 用于枚举 lazy VMA、部分失败、远端 TLB、OOM drain、锁顺序和 PPN 所有权等边界。实验参数、回退门槛和最终合入仍以维护者判断为准。

AI 也会提出无效方案。page cache reverse index、child index、TLSF 小对象池和扩大预读等方向都曾在局部上看起来合理，但完整轮没有收益或发生正确性问题。保留这些反例，是为了说明我们没有把生成的建议当成既定路线，也没有只挑选对结论有利的数据。

5.2 复现环境与证据分级

建议使用与记录相同的 QEMU 9.2.1、RISC-V virt、8 vCPU、16 GiB guest 内存、final 内核、同一赛题镜像和 -snapshot。A/B 两侧只改变待测提交或 feature，不能在中途更换镜像、QEMU 参数、宿主 CPU 集合或 workload。对于跨架构公共代码，还应在同一提交上完成 RISC-V 和 LoongArch 检查。

实验记录按证据强度分为三类：

1. **完整 A/B**：具有相同条件、完整成功标记和相邻对照，可用于判断墙钟收益；
2. **固定窗口诊断**：pc-hot、wait-hot 或 syscall profile 到达固定时限，用于解释热点与机制；
3. **机制统计**：命中率、变化摘要、锁等待或 refill 来源，用于确认代码按设计工作。

后两类不能单独写成“性能提升”。如果固定窗口中候选执行了更多指令，可能只是推进了更多 workload；如果缓存命中率很高，也仍可能因锁竞争和淘汰开销而变慢。

5.3 pc-hot 与 syscall profile 的复现方法

pc-hot 需要与内核匹配的 ELF 符号。下面给出最小流程，QEMU 后面的磁盘和网络参数应与正式 runner 保持一致：

```
cd os
./scripts/pc-hot/pc-hot-rv.sh build
./scripts/pc-hot/pc-hot-rv.sh run /tmp/pcs-rv.txt -- \
    timeout 300 qemu-system-riscv64 -machine virt \
    -kernel ./kernel-rv-final -m 16G -smp 8 -nographic ...
./scripts/pc-hot/pc-hot-rv.sh analyze \
    /tmp/pcs-rv.txt ./kernel-rv-final 50
```

分析候选和对照时，输出文件必须分开，ELF 也不能混用。除了 top symbols，还应记录总指令、每个 vCPU 分布和 workload 到达位置。需要区分计算与等待时，可在相同参数下换用 wait-hot-rv.sh。

syscall profile 的 RISC-V full-system 入口如下：

```
./scripts/syscall-profile/syscall-profile-rv.sh build
./scripts/syscall-profile/syscall-profile-rv.sh run \
    /tmp/syscalls.txt backend=auto paths=1 \
    max_path=256 top_paths=200 -- \
    timeout 300 qemu-system-riscv64 -machine virt \
    -kernel ./kernel-rv-final -m 16G -smp 8 -nographic ...
./scripts/syscall-profile/analyze.py \
    /tmp/syscalls.txt --top 30 > /tmp/syscalls.md
```

full-system 下应确认 backend=auto 实际选择 ecall。强制使用 QEMU native syscall callback 在 WaterOS full-system 实测为零事件，因为 syscall 由 guest 内核处理；该后端主要用于 linux-user 对照。

5.4 完整 BuildStorm A/B

完整复现可按以下顺序进行：

1. 为 baseline 和 candidate 分别记录 git rev-parse HEAD，构建 final kernel 并保存 SHA-256；
2. 执行 make rv_check 与 make la_check，再运行改动涉及的功能测例；
3. 固定宿主 CPU 亲和性、QEMU 9.2.1、镜像哈希、SMP 与内存，确认磁盘使用 -snapshot；

4. 交错运行完整 A/B，至少取得两轮候选和相邻对照；
5. 检查 toolchain、minibuild 和 multi compile 标记，不只提取最后一个数字；
6. 计算各自中位数与相对变化，并把 timeout、panic、SIGSEGV、OOM 或缺项记为失败，而不是删除样本；
7. 对超过门槛的候选再运行 pc-hot 或机制统计，确认没有把开销转移到其他子系统。

syscall-chain 任务将大改动拆成可回滚的小提交：测量 runner、mount namespace、dentry、FD slot、I/O lease、lookup token、exec 稳定句柄、lazy ELF、mmap/munmap 变化摘要、只读路径和 futex 分片分别验收。复现时可沿这些提交定位行为变化，但不能把只有静态检查、尚无 QEMU A/B 的任务写成已确认性能收益。

5.5 文档和实现如何对应

性能任务索引记录了早期 benchmark 缺口、风险级别和 FS/MM/allocator 的候选；独立实验文档保存工具设计、固定条件、命中率和回退原因；syscall-chain 简报保存每个接口改动的语义边界。perf/* 分支则保留代码和失败实验。本文把这些材料按因果链重新组织，数字仍应回到对应实验记录核对。

若后续修改正文中的结果，至少同时更新：被测提交与参数、结果表中的证据类型、失败样本、最终内核上的重新验证状态。这样可以避免文档随着主线演进而留下无法复现的“历史最好值”。

第 6 章 总结与下一步工作

这轮优化最重要的成果并不是某一个百分比，而是建立了一条可以继续使用的分析路径。我们先用完整标记确认 workload 真实完成，再用 pc-hot 看指令分布、wait-hot 看核心空闲与阻塞、syscall profile 看 ABI 调用形态；得到机制假设后只改变一个主要因素，最后回到固定条件的完整 A/B。这个过程使一些局部漂亮但端到端无效的方案能够及时回退。

最终保留下来的改动也有清晰共同点。syscall 和 VFS 复用一次调用中已经取得的参数、fd 与 lookup 结果；文件系统缓存根据实际复用决定准入和容量；ELF 与只读 mmap 在不同进程间共享稳定物理页；mprotect、mmap、munmap 和 COW 只对真实页表变化执行必要同步；匿名页和页表页的清零则由 idle CPU 提前完成。它们没有改变用户可见 ABI，而是减少了同一语义的重复实现成本。

现有结果仍有边界。历史实验跨越多个 main 基线，不能把独立百分比相加；部分结果只有单轮完整记录；LoongArch 与 RISC-V 的 Linux 对照条件尚未完全统一。因此，最终提交前仍应在固定宿主 CPU、相同 QEMU 和镜像上重跑当前 main，给出 WaterOS 与 Linux 的统一基线、多轮中位数和离散程度。

下一步优先关注三项工作。第一，重新 profile 当前主线，确认只读页共享和零页池之后的新热点，避免继续优化已经消失的路径。第二，检查 rustc 关键阶段的单核瓶颈与 vCPU 不均衡，区分 scheduler、锁等待和 guest 本身串行阶段。第三，在正确性可控的前提下研究 VirtIO 多请求并行与更精确的 TLB range shutdown；历史分支中相关方案已有失败记录，新的实现必须先解释中断、完成队列和错误恢复边界。

性能优化不会在“找到最热函数”时结束。真正可维护的结果应当同时满足三个条件：能够从 workload 数据解释根因，能够在接口层说明正确性边界，并能够在完整运行中重复观察到收益。本文记录的工具、反例和复现方法，就是为了让后续工作仍然遵守这三个条件。